

4 Ordinamento (cont.)

Ordinamento Finora abbiamo visto due algoritmi di ordinamento, in cui avevamo le seguenti premesse:

IN: $a_1 \dots a_n$;

OUT: permutazione $a'_1 \dots a'_n$ ordinata.

In particolare, abbiamo concluso che:

- InsertionSort: $O(n^2)$, basato su scambi;
- MergeSort: $\Theta(n \log n)$, ma con un costo in termini di **memoria**.

Memoria

- InsertionSort:

$input + 1$ variabile $\Rightarrow$ spazio **costante** $\Theta(1)$ (detto “in loco”)

- MergeSort: spazio con costo lineare.

$$\begin{aligned} S_{MS}(n) &= \max \left\{ S\left(\left\lfloor \frac{n}{2} \right\rfloor\right), S\left(\left\lceil \frac{n}{2} \right\rceil\right), \Theta(n) \right\} \\ &= \Theta(n) \end{aligned}$$

4.1 Heapsort

L'**Heapsort**¹ è un algoritmo di ordinamento basato su una struttura chiamata **heap**, che prende le caratteristiche positive di InsertionSort e MergeSort:

- in “loco” (spazio $\Theta(1)$);
- complessità $\Theta(n \log n)$.

Cos'è un heap? Un **heap** è una struttura dati basata sugli alberi che soddisfa la “proprietà di heap”: se A è un genitore di B, allora la chiave di A è ordinata rispetto alla chiave di B conformemente alla relazione d'ordine applicata all'intero heap.

Seguono alcune definizioni.

¹Anche qui, si consiglia di dare un occhio ad altre fonti. In classe, sono stati viste molte rappresentazioni grafiche degli heap, e, come già detto, in $\text{\LaTeX}$ non è per me facile rappresentarli.

Altezza: è la distanza dalla radice alla foglia più distante;

Albero completo: è un albero di altezza h con $\sum_{i=0}^h 2^i - 1$ nodi;

Albero quasi completo: è un albero completo a tutti i livelli eccetto l'ultimo, in cui possono mancare delle foglie e le foglie presenti sono addossate a sinistra.

Gli heap verranno rappresentati in array monodimensionali, nel modo descritto di seguito:

$$\forall i > 0$$

- $A[i]$ è il nodo genitore;
- $A[2i]$ è il figlio sx del nodo $A[i]$;
- $A[2i+1]$ è il figlio dx.

Inoltre, ogni array A sarà dinamico, e avrà:

- $A.length$ potenziale spazio, capacità massima dell'array;
- $A.heapsize$ celle effettive dell'array.

Vediamo alcune funzioni di utilità che verranno usate.

LEFT(i)

```
// restituisce il figlio sx del nodo i
1 return 2 * i
```

RIGHT(i)

```
// restituisce il figlio dx del nodo i
1 return 2 * i + 1
```

PARENT(i)

```
// restituisce il genitore del nodo i
1 return  $\lfloor i/2 \rfloor$ 
```

4.1.1 Max Heap

Max Heap è uno heap che soddisfa la seguente proprietà:

$$\begin{aligned} & \forall \text{ nodo } A[i], \\ & A[i] \geq \text{discendenti} \\ & \Downarrow \\ & A[i] \geq A[\mathbf{Left}(i)], A[\mathbf{Right}(i)] \end{aligned}$$

Equivalentemente

$$\begin{aligned} & \forall \text{ nodo } A[i], \\ & A[i] \leq \text{antenati} \\ & \Downarrow \\ & A[i] \leq A[\mathbf{Parent}(i)] \end{aligned}$$

Osservazioni

- Uno heap con un solo elemento è un **Max Heap**.
- Dati due Max Heap T_1 e T_2 e un nodo N , possiamo “combinarli” in uno heap con N come radice, T_1 come **left** e T_2 come **right**.

Ecco ora una procedura che, dato un nodo i , trasforma in un Max Heap il sotto-albero eradicato in esso (con radice i).

MAX-HEAPIFY(A, i)

```

1   $l = \mathbf{LEFT}(i)$ 
2   $r = \mathbf{RIGHT}(i)$ 
3  if ( $l \leq A.heapsize$ ) and ( $A[l] > A[i]$ )
4       $max = l$ 
5  else
6       $max = i$ 
7  if ( $r \leq A.heapsize$ ) and ( $A[r] > A[max]$ )
8       $max = r$ 
9  if ( $max \neq i$ )
10      $A[i] \leftrightarrow A[max]$ 
11     MAX-HEAPIFY( $A, max$ )
```

Correttezza di MaxHeapify

- **Casi base:** $max = i$, distunguo due casi:
 - i è foglia ($l, r > A.heapsize$);
 - $A[i] \geq A[l], A[r]$;
- **Induzione:** $max \neq i$, distunguo due casi:
 - $max = l, A[l] \geq A[i], A[r]$;
 - $max = r, A[r] \geq A[i], A[l]$.

Complessità $O(h)$, con h altezza del sotto-albero radicato in i

$$\begin{aligned}
 n &\geq (2^{(h-1)+1}) + 1 = 2^h \\
 h &\leq \log_2 n \\
 &\Rightarrow O(h) \cong O(\log n)
 \end{aligned}$$

Ora vogliamo scrivere una procedura che costruisce un **Max Heap** da un array qualunque.

Quali sono i nodi foglia?

- Se $i \geq \lfloor \frac{n}{2} \rfloor + 1$

$$\begin{aligned}
 2i &= 2\left(\frac{n}{2} + 1\right) \geq n + 2 - 1 = n + 1 \\
 &\Rightarrow i \text{ foglia}
 \end{aligned}$$

- Se $i \leq \lfloor \frac{n}{2} \rfloor$

$$\begin{aligned}
 2i &= 2\lfloor \frac{n}{2} \rfloor \leq n \\
 &\Rightarrow i \text{ non foglia}
 \end{aligned}$$

BUILD-MAX-HEAP(A)

```

1   $A.heapsize = A.length$ 
2  for  $i = \lfloor A.length/2 \rfloor$  down to 1
3      MAX-HEAPIFY( $A, i$ )

```

L'algoritmo esegue $\frac{n}{2}$ volte **MaxHeapify** (che ha complessità $O(\log n)$), ottenendo una complessità finale $O(n \log n)$, tuttavia questa stima è molto pessimistica.

Definiamo:

- h_T altezza del cammino più lungo dello heap;
- $h_T - 1$ di conseguenza è l'altezza dell'albero meno l'ultimo livello, che è generalmente incompleto.

$$\begin{aligned}
 n &= \left(2^{(h_T-1)+1} - 1\right) + 1 \\
 &= 2^{h_T} \\
 h_T &\leq \log n \\
 n &\geq 2^{h_T}
 \end{aligned}$$

$$\begin{aligned}
 T(n) &= \sum_{h=1}^{\lfloor \log n \rfloor} 2^{h_T-h} \cdot O(h) \\
 &\quad (2^{h_T-h} = \# \text{ chiamate a MaxHeapify al livello } h) \\
 &= \sum_{h=1}^{\lfloor \log n \rfloor} \frac{2^{h_T}}{2^h} O(h) \quad (2^{h_T} = n) \\
 &= O\left(\left(\sum_{h=1}^{\lfloor \log n \rfloor} \frac{h}{2^h}\right)n\right) = O(n) \quad \left(\sum_{h=1}^{\lfloor \log n \rfloor} \frac{h}{2^h} \leq \sum_{h=1}^{\infty} \frac{h}{2^h} = \frac{\frac{1}{2}}{(1 - \frac{1}{2})^2} = 2\right)
 \end{aligned}$$

Passiamo ora all'algoritmo di ordinamento **Heapsort**. La radice di un **Max Heap** contiene il valore massimo. Quindi, la prima operazione, e quella su cui si basa **Heapsort**, consiste nel mettere la radice in ultima posizione.

Es. A: 9 8 7 5 7 4 0 4 3 6 1 2 è un max heap.
 $\Rightarrow$ 8 7 5 7 4 0 4 3 6 1 2 9 ignoro l'ultimo elemento, chiamo
MaxHeapify sulla radice e itero.

Poi chiama **MaxHeapify** sul resto dell'array per renderlo un **Max Heap**, e itera il procedimento sul nuovo array.

HEAPSORT(A)

```

1  BUILD-MAX-HEAP( $A$ ) //  $O(n)$ 
2  for  $i = A.length$  down to 2
3       $A[1] \leftrightarrow A[i]$ 
4       $A.heapsize = A.heapsize - 1$ 
5      MAX-HEAPIFY( $A, 1$ ) //  $O(\log n)$ 
```

Complessità $O(n \log n)$.

4.1.2 Code con Priorità

S insieme dinamico di oggetti.

x è l'indice, $x.key$ è il corrispondente valore relativo a quell'indice. Voglio poter eseguire le seguenti operazioni:

- `Insert( $S, x$ )`
- `Max( $S$ )`
- `ExtractMax( $S$ )`
- `IncreaseKey( $S, x, \delta$ )`
- `ChangeKey( $S, x, \delta$ )`
- `Delete( $S, x$ )`

Idea Uso un **Max Heap** (A).

`MAX( $A$ )`

```
1  if  $A.heapsize = 0$ 
2      error
3  else return  $A[1]$ 
```

La procedura `Max( $A$ )` ha complessità costante $\Theta(1)$.

`EXTRACT-MAX( $A$ )`

```
1   $max = A[1]$ 
2   $A[1] = A[A.heapsize]$ 
3   $A.heapsize = A.heapsize - 1$ 
4  MAX-HEAPIFY( $A, 1$ ) // ripristina le proprietà di MaxHeap
5  return  $max$ 
```

La procedura `ExtractMax( $A$ )` ha la stessa complessità di `MaxHeapify`: $O(\log n)$.

Per `Insert`, le cose diventano più delicate. L'idea è quella di inserire in coda ad A : in questo modo, l'unico elemento che potrebbe compromettere la proprietà di **Max Heap** è la cella di indice i (nel nostro caso, l'ultima). Deve valere la proprietà:

$$\begin{aligned} &\text{Per ogni } j \neq i \\ &A[j] \leq \text{antenati} \end{aligned}$$

Non possiamo dire nulla su i . Va ristabilita la proprietà di **Max Heap**: per fare ciò usiamo la procedura `MaxHeapifyUp`.

MAX-HEAPIFY-UP(A, i)

```

1  if ( $i > 1$ ) and ( $A[i] > A[\text{PARENT}(i)]$ )
2       $A[i] \leftrightarrow A[\text{PARENT}(i)]$ 
3      MAX-HEAPIFY-UP( $A, \text{PARENT}(i)$ )

```

Correttezza di MaxHeapifyUp

Casi base

($i = 1$) ok, non faccio nulla;

($A[i] \leq A[\text{Parent}(i)]$) ok, la proprietà di **Max Heap** è mantenuta.

Induzione

($A[i] > A[\text{Parent}(i)]$) scambio le due celle. I discendenti (sottoalberi) della nuova cella $A[i]$ mantengono la proprietà di **Max Heap**.

Complessità $O(\log i)$, nel caso peggiore $O(\log n)$.

Ecco ora lo pseudocodice della funzione **Insert**.

INSERT(A, x)

```

1   $A.\text{heapsize} = A.\text{heapsize} + 1$ 
2   $A[A.\text{heapsize}] = x$ 
3  MAX-HEAPIFY-UP( $A, A.\text{heapsize}$ )

```

Insert ha complessità $O(\log n)$, la stessa di **MaxHeapifyUp**.

INCREASE-KEY(A, i, δ)

// **Precondizione:** $\delta \geq 0$

```

1   $A[i] = A[i] + \delta$ 
2  MAX-HEAPIFY-UP( $A, i$ )

```

IncreaseKey ha complessità $O(\log n)$.

CHANGE-KEY(A, i, δ)

```

1   $A[i] = A[i] + \delta$ 
2  if  $\delta > 0$ 
3      MAX-HEAPIFY-UP( $A, i$ )
4  else //  $\delta \leq 0$ 
5      MAX-HEAPIFY( $A, i$ )

```

ChangeKey è come **IncreaseKey**, ma può utilizzare valori di δ qualsiasi, ed è corretto per la seguente proprietà:

Se per ogni $j \neq i$ $A[j] \geq$ discendenti
 $\Rightarrow$ dopo **MaxHeapify** ho un **MaxHeap**

```
DELETE( $A, i$ )
1   $old = A[i]$ 
2   $A[i] = A[A.heapsize]$ 
3   $A.heapsize = A.heapsize - 1$ 
4  if  $old \leq A[i]$ 
5      MAX-HEAPIFY-UP( $A, i$ )
6  else
7      MAX-HEAPIFY( $A, i$ )
```

Delete ha complessità $O(\log n)$.